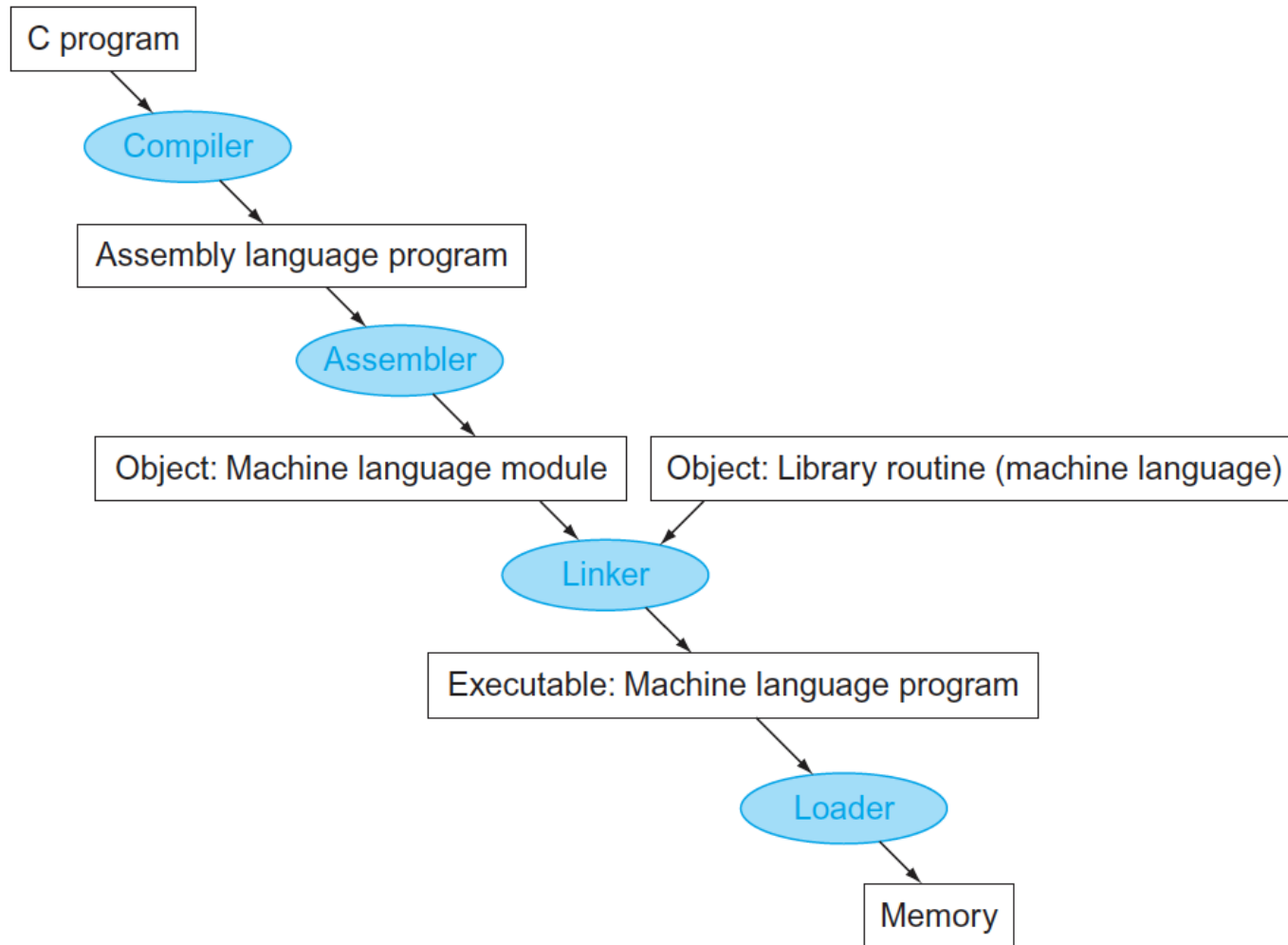


Arquitetura de Computadores

Tópico 1 – Arquitetura MIPS

Traduzindo e Iniciando um Programa



Sumário



- Introdução
- Registradores e Memória
- Tipos de Dados
- Instruções
- Modos de Endereçamento
- Formato das Instruções

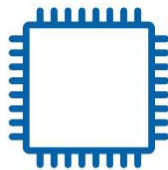
Wave Computing Signs New License Agreement with MediaTek

Global Fabless Semiconductor Leader is Leveraging Wave Computing's MIPS Processors to Power System-on-Chip (SoC) Designs for Mobile, Home Entertainment and IoT Devices

[LEARN MORE](#)

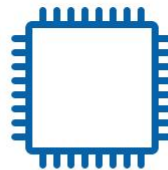
MIPS Processors

Widely used and backed by an active ecosystem of hardware and software partners, MIPS processors are the CPU of choice for the future of computing.



Current Cores

Latest family of MIPS CPUs offers best-in-class performance, power and area efficiency



Classic Cores

Widely licensed and cost-effective solutions for embedded and multimedia applications



MIPS Architecture

Highest levels of performance with clean, elegant design

Sumário



- Introdução
- **Registradores e Memória**
- Tipos de Dados
- Instruções
- Modos de Endereçamento
- Formato das Instruções

Registradores

- 32 registradores de propósito geral de 32 bits
 - \$0, \$1, ..., \$31
 - operações inteiras
- \$0 tem sempre valor 0
- \$31 guarda endereço de retorno de sub-rotina

Organização da memória

- Endereçada à BYTE ou à PALAVRA
 - Como sabemos?
 - Qual a diferença entre uma memória mapeada a byte e a palavra?
- Uma palavra no MIPS tem 32 bits, ou 4 bytes

Registradores e Memória

- Os operandos das instruções aritméticas e lógicas têm de estar em registradores
- O compilador associa as variáveis com registradores
- E programas com muitas variáveis?
 - Usar load/store pra transferir dados da memória pros registradores
 - Se não houver registradores suficientes
 - SPILL (derramamento) de registradores pra memória

Sumário



- Introdução
- Registradores
- **Tipos de Dados**
- Instruções
- Modos de Endereçamento
- Formato das Instruções

Tipos de dados

- Dados inteiros disponíveis em instruções load e store
 - bytes
 - meias-palavras de 16 bits
 - palavras de 32 bits
- Dados inteiros disponíveis em instruções aritméticas e lógicas
 - meias-palavras de 16 bits (estendidos para 32 bits)
 - palavras de 32 bits

Sumário



- Introdução
- Registradores
- Tipos de Dados
- **Instruções**
- Modos de Endereçamento
- Formato das Instruções

Instruções Aritméticas e Lógicas

- **Instruções Soma e Subtração**
 - add, sub, and, or, nor, xor

ADD Rd, Rt, Rs
(Rd $\leftarrow$ Rt+Rs)

Código C: A = B + C + D;
 E = F - A;

Código MIPS

Instruções Aritméticas e Lógicas



- Instruções de deslocamento variável

- Número de bits a deslocar está em um registrador
- sllv, srlv – shift lógico (entra 0 na extremidade)
- srav – shift aritmético (duplica sinal)

- Exemplo:

- `SRAV Rd, Rt, Rs` ($Rd \leftarrow Rt$) deslocamento em Rs

- Instruções de deslocamento constante

- Número de bits a deslocar está no campo shamt (imediato)
- sll, sra, srl

- Exemplo:

- `SLL Rd, Rt, shamt` ($Rd \leftarrow Rt$) deslocamento no campo shamt

Exemplo

- Considere que os operandos estão associados, da esquerda para direita, em
 - \$s1, \$s2, \$s3, \$s4, \$s5

Código em C

$$X = ((A + B) + (C - D))$$

Código MIPS

Instruções Load/Store

- **Instrução Load Word**

- Primeiro operando é o destino

`LW Rt, imed16(Rs)`

`Rt <- Mem[est(imed16) + Rs]`

- endereçamento sempre por base e deslocamento

Instruções Load/Store



- **Instrução Store Word**

- Primeiro operando é fonte
`SW Rt, imed16(Rs)`

`Mem[est(imed16) + Rs] <- Rt`

- endereçamento sempre por base e deslocamento

Exemplo

Código C :

```
A[8] = h + A[8];
```

Código MIPS:

Instruções de Desvio Condicional

- **Instruções Bne e Beq**

- Instruções que comparam dois registradores

`bne rs, rt, Label`

`(PC = PC + (Imed16 com sinal estendido<<2), se rs!=rt)`

Código em C:

```
if (i==j) h = i + j;
```

Código em MIPS:

Instruções de Desvio Condicional

- **PSEUDO Instruções bgez, bgtz, blez, bltz**

- Instruções comparam um registrador com o valor 0

`BGEZ $t0, Label`

(PC = PC + (Imed16 com sinal estendido<<2), se $\$t0 \geq 0$)

Código em C: `if (i<0) h = i + j;`

Código em MIPS:

Instruções de Desvio Incondicional

- **Instrução j**

- Endereço destino é imediato concatenado com o PC)

- `J imed26`

`(PC <- PC[31:28] & Imed26 & "00")`

- **Instrução jal (jump and link)**

- `Jal imed26`

`($ra <- PC e PC <- PC[31:28] & Imed26 & "00")`

- Serve para chamadas de sub-rotina,
- endereço de retorno salvo em \$31

- **Instrução jr**

- Endereço destino contido em registrador

- `Jr $ra (PC <- $ra)`

- Serve para **retornar** de sub-rotina (Registrador?)

Exemplo

Código em C:

```
if (i != j)
    h = i + j;
else
    h = i - j;
```

Código em MIPS

)
)

Exemplo

Código em C:

```
while (save[i] == k) i = i + j;
```

Código em MIPS:

Chamadas de Função

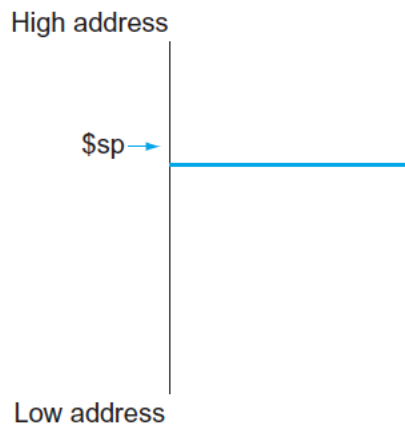
- Pela notação, registradores:
 - \$a0-\$a3 – usados para passagem de parâmetros
 - \$v0-\$v1 – usados para retornar de valor
- Código em C

```
int leaf_example (int g, int h, int i, int j)
{
    int f;

    f = (g + h) - (i + j);
    return f;
}
```

Chamadas de Função

- Pela notação, registradores:
 - \$a0-\$a3 – usados para passagem de parâmetros
 - \$v0-\$v1 – usados para retornar de valor
- Código em MIPS



```
leaf_example: addi $sp, $sp, -12 # adjust stack to make room for 3 items
               sw  $t1, 8($sp)   # save register $t1 for use afterwards
               sw  $t0, 4($sp)   # save register $t0 for use afterwards
               sw  $s0, 0($sp)   # save register $s0 for use afterwards

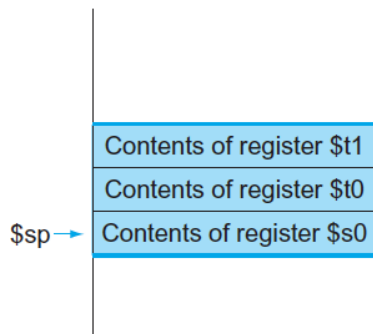
               add $t0,$a0,$a1 # register $t0 contains g + h
               add $t1,$a2,$a3 # register $t1 contains i + j
               sub $s0,$t0,$t1 # f = $t0 - $t1, which is (g + h)-(i + j)
               add $v0,$s0,$zero # returns f ($v0 = $s0 + 0)

               lw  $s0, 0($sp)   # restore register $s0 for caller
               lw  $t0, 4($sp)   # restore register $t0 for caller
               lw  $t1, 8($sp)   # restore register $t1 for caller
               addi $sp,$sp,12 # adjust stack to delete 3 items
               jr   $ra          # jump back to calling routine
```



Chamadas de Função

- Pela notação, registradores:
 - \$a0-\$a3 – usados para passagem de parâmetros
 - \$v0-\$v1 – usados para retornar de valor
- Código em MIPS

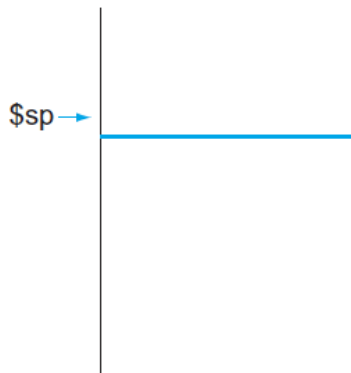


```
leaf_example: addi $sp, $sp, -12 # adjust stack to make room for 3 items
              sw  $t1, 8($sp)    # save register $t1 for use afterwards
              sw  $t0, 4($sp)    # save register $t0 for use afterwards
              sw  $s0, 0($sp)    # save register $s0 for use afterwards
              add $t0,$a0,$a1 # register $t0 contains g + h
              add $t1,$a2,$a3 # register $t1 contains i + j
              sub $s0,$t0,$t1 # f = $t0 - $t1, which is (g + h)-(i + j)
              add $v0,$s0,$zero # returns f ($v0 = $s0 + 0)
              lw  $s0, 0($sp)    # restore register $s0 for caller
              lw  $t0, 4($sp)    # restore register $t0 for caller
              lw  $t1, 8($sp)    # restore register $t1 for caller
              addi $sp,$sp,12 # adjust stack to delete 3 items
              jr   $ra          # jump back to calling routine
```



Chamadas de Função

- Pela notação, registradores:
 - \$a0-\$a3 – usados para passagem de parâmetros
 - \$v0-\$v1 – usados para retornar de valor
- Código em MIPS



```
leaf_example: addi $sp, $sp, -12 # adjust stack to make room for 3 items
              sw  $t1, 8($sp)    # save register $t1 for use afterwards
              sw  $t0, 4($sp)    # save register $t0 for use afterwards
              sw  $s0, 0($sp)    # save register $s0 for use afterwards
              add $t0,$a0,$a1 # register $t0 contains g + h
              add $t1,$a2,$a3 # register $t1 contains i + j
              sub $s0,$t0,$t1 # f = $t0 - $t1, which is (g + h)-(i + j)
              add $v0,$s0,$zero # returns f ($v0 = $s0 + 0)
              lw  $s0, 0($sp)    # restore register $s0 for caller
              lw  $t0, 4($sp)    # restore register $t0 for caller
              lw  $t1, 8($sp)    # restore register $t1 for caller
              addi $sp,$sp,12 # adjust stack to delete 3 items
              jr   $ra          # jump back to calling routine
```



Chamadas de Função Aninhadas

- Pela notação, registradores:
 - \$a0-\$a3 – usados para passagem de parâmetros
 - \$v0-\$v1 – usados para retornar de valor
- Código em C
 - \$a0 corresponde ao n

```
int fact (int n)
{
    if (n < 1) return (1);
    else return (n * fact(n - 1));
}
```

Chamadas de Função Aninhadas

- Pela notação, registradores:
 - \$a0-\$a3 – usados para passagem de parâmetros
 - \$v0-\$v1 – usados para retornar de valor
- Código em MIPS

fact:

```
addi    $sp, $sp, -8 # adjust stack for 2 items
sw      $ra, 4($sp)  # save the return address
sw      $a0, 0($sp)  # save the argument n
slti    $t0, $a0, 1  # test for n < 1
beq     $t0, $zero, L1 # if n >= 1, go to L1
addi    $v0, $zero, 1 # return 1
addi    $sp, $sp, 8  # pop 2 items off stack
jr      $ra          # return to caller
```

```
L1: addi $a0, $a0, -1 # n >= 1: argument gets (n - 1)
jal fact             # call fact with (n - 1)
```

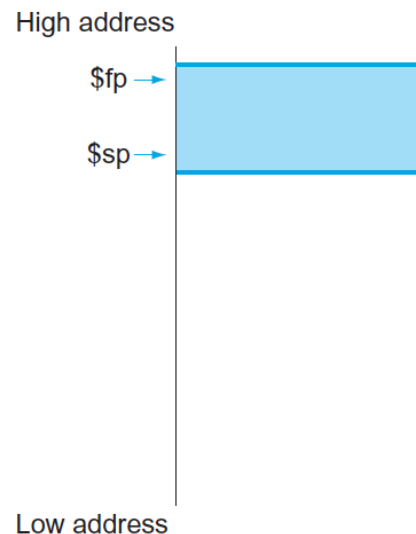
Chamadas de Função Aninhadas

- Pela notação, registradores:
 - \$a0-\$a3 – usados para passagem de parâmetros
 - \$v0-\$v1 – usados para retornar de valor
- Código em MIPS

```
lw    $a0, 0($sp)  # return from jal: restore argument n
lw    $ra, 4($sp)  # restore the return address
addi  $sp, $sp, 8   # adjust stack pointer to pop 2 items
mul   $v0,$a0,$v0   # return n * fact (n - 1)
jr    $ra           # return to the caller
```

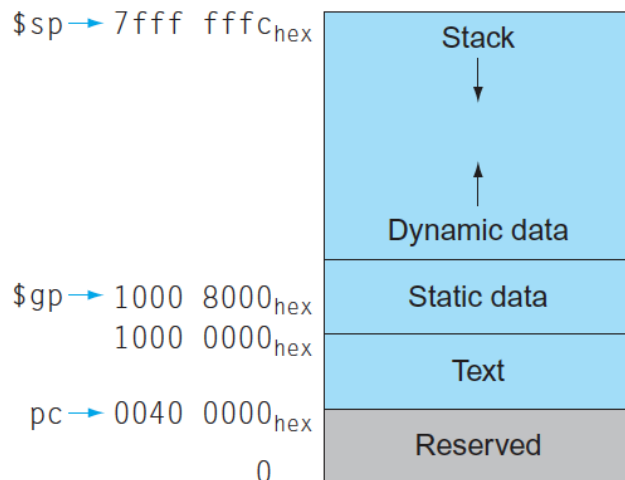
Alocando Espaço para Dados na Pilha

- A pilha também é utilizada para armazenar dados locais de funções mas que não cabem nos registradores:
 - Estruturas, arrays, etc
 - Este espaço é chamado de procedure frame (PF)
 - Para acessar estes dados na pilha é utilizado o registrador frame pointer (\$fp), aponta para a primeira palavra do PF



Alocando Dinamicamente Dados

- A heap é utilizada para armazenar dados alocados dinamicamente
 - malloc em C, new em Java,
 - Para acessar a heap é utilizado o registrador global pointer (\$gp), que também é utilizado para acessar dados estáticos



- \$gp inicializado em 0x1000 8000 mas pode acessar dados de 0x1000 0000 até 0x1000 FFFF
- São usados os 16 bits de offset de LW e SW para acessar esta faixa, pode ser positivo ou negativo

Variáveis Automáticas e Estáticas

- Por definição, variáveis:
 - Automáticas são locais à função e descartadas quando a função termina sua execução
 - Estáticas são declaradas fora de funções tendo escopo global e acessíveis em toda a execução do programa
 - Para realizar o acesso mais simples à essas variáveis é utilizado o registrador global pointer (\$gp)

- **Instruções Lógicas e Aritméticas com Imediatos**

- Pequenas constantes são utilizadas muito frequentemente

$A = A + 5;$

`addi $s4, $s3, 5` (constantes de 16 bits)

`B++;`

`addi $s4, $s4, 1` (constantes de 16 bits)

Outros exemplos

- `andi, ori, xori, slti`

Movimentação de dados

- \$zero usado para sintetizar operações populares
 - Carga de constante = soma imediata onde \$zero é um dos operandos

```
addi    $s5, $zero, 10
```

- Mover de registrador para registrador = soma com \$0

```
add     $s6, $s2, $zero
```

Sumário



- Introdução
- Registradores
- Tipos de Dados
- Instruções
- **Modos de Endereçamento**
- Formato das Instruções

Modos de endereçamento

- **Modo registrador**

- para instruções aritméticas e lógicas: operando está em registrador
- para instruções de desvio incondicional: endereço está em registrador (Jump Register)

2. Register addressing

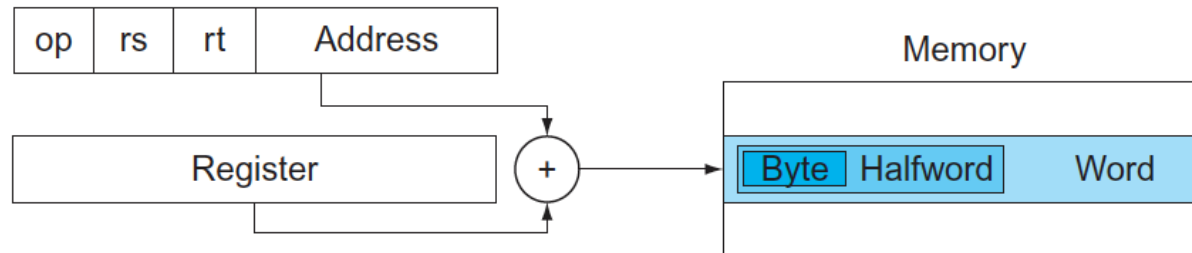


Modos de endereçamento

- **Modo base**

- para instruções load e store
- base é registrador inteiro de 32 bits
- deslocamento de 16 bits contido na própria instrução

3. Base addressing

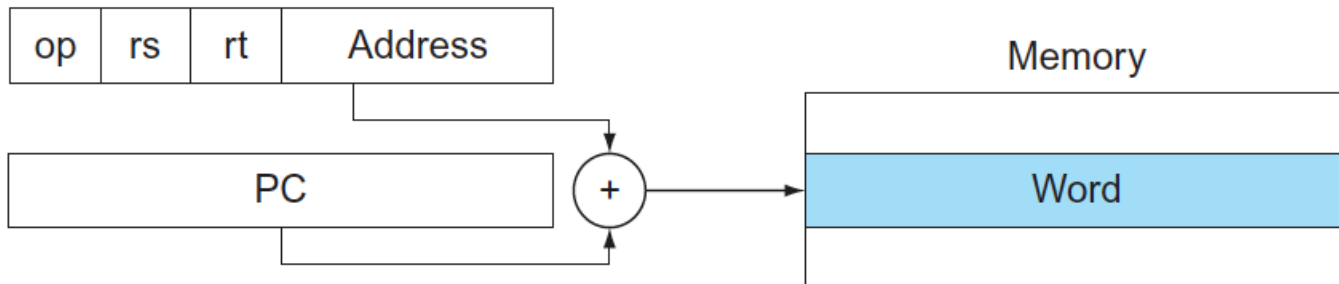


Modos de endereçamento

- **Modo relativo ao PC**

- para instruções de branch condicional
- endereço é a soma do PC com deslocamento contido na instrução
- deslocamento é dado em palavras e precisa ser multiplicado por 4

4. PC-relative addressing



Modos de endereçamento

- **Modo imediato**

- para instruções aritméticas e lógicas com imediato
- dado imediato de 16 bits contido na própria instrução
- dado é estendido para 32 bits
 - extensão com sinal nas instruções aritméticas
 - extensão sem sinal nas instruções lógicas

1. Immediate addressing

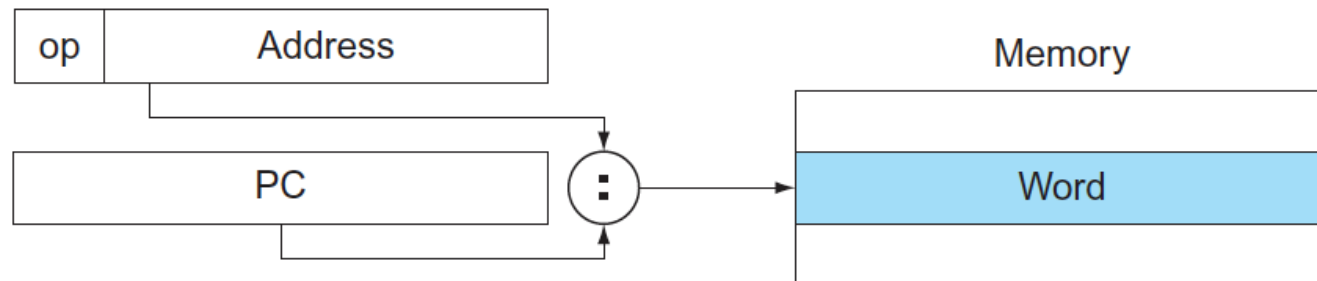


Modos de endereçamento

- **Modo pseudo-direto**

- para instruções de desvio incondicional
- instrução tem campo com endereço de palavra com 26 bits
- 4 bits mais significativos obtidos do PC
- só permite desvios dentro de uma área de 256 Mbytes

5. Pseudodirect addressing



Sumário



- Introdução
- Registradores
- Tipos de Dados
- Instruções
- Modos de Endereçamento
- **Formato das Instruções**

Formatos das instruções

- Instruções têm 32 bits
- Todas têm opcode de 6 bits
- Apenas três formatos diferentes
 - R
 - I
 - J

Formatos das instruções

- **Instruções de tipo R**

- aritméticas e lógicas
- movimentação entre registradores

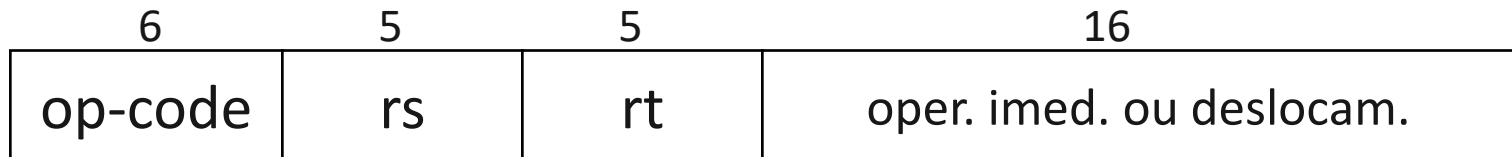


- op: operação básica da instrução, “opcode”
- rs: 1o registrador de origem
- rt: 2o registrador de origem
- rd: registrador de destino
- shamt: *shift amount*
- funct: Função, ou function code, indica a função específica que a ALU irá fazer

Formatos das instruções

- **Instruções de tipo I**

- loads, stores
- operações aritméticas e lógicas com operando imediato
- desvios condicionais (“branches”)
- desvios incondicionais para endereço em registrador



Formatos das instruções

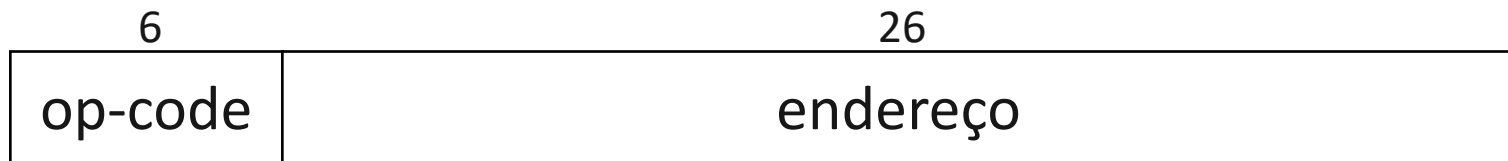
	6	5	5	5	5	6
R	op-code	rs	rt	rd	shamt	funct
I	op-code	rs	rt	oper. imed. ou deslocam.		

- Primeiros 3 campos são os mesmos do R
 - Facilita para o hardware decodificar
- Porém, o campo `rt` muda de significado
 - Formato R: `rt` é a 2a origem
 - Formato I: `rt` pode ser destino

Formatos das instruções

- **Instruções de tipo J**

- desvios com endereçamento absoluto
- chamada de sub-rotina



Três formatos

	6	5	5	5	5	6
R	op-code	rs	rt	rd	shamt	funct
I	op-code	rs	rt	oper. imed. ou deslocam.		
J	op-code	endereço				